

Database Schema

HEMAS

Filter objects

hospital
lab
market
retail_d
sakila
sys
world

Filter Rows: Limit to 1000 rows

1 • USE marketing_campaign;
2 • DESCRIBE marketing_campaign_cleaned;
3
4

Result Grid

Field	Type	Null	Key	Default	Extra
ID	int	YES		HULL	
Year_Birth	int	YES		HULL	
Education	text	YES		HULL	
Marital_Status	text	YES		HULL	
Income	double	YES		HULL	
Kidhome	int	YES		HULL	
Teenhome	int	YES		HULL	
Dt_Customer	text	YES		HULL	
Recency	int	YES		HULL	
MntWines	int	YES		HULL	
MntFruits	int	YES		HULL	
MntMeatProd...	int	YES		HULL	
MntFishProdu...	int	YES		HULL	
MntSweetPro...	int	YES		HULL	
MntGoldProds	int	YES		HULL	
NumDealsPur...	int	YES		HULL	
NumWebPurc...	int	YES		HULL	
NumCatalogP...	int	YES		HULL	
NumStorePur...	int	YES		HULL	
NumWebVisit...	int	YES		HULL	
AcceptedCmp3	int	YES		HULL	

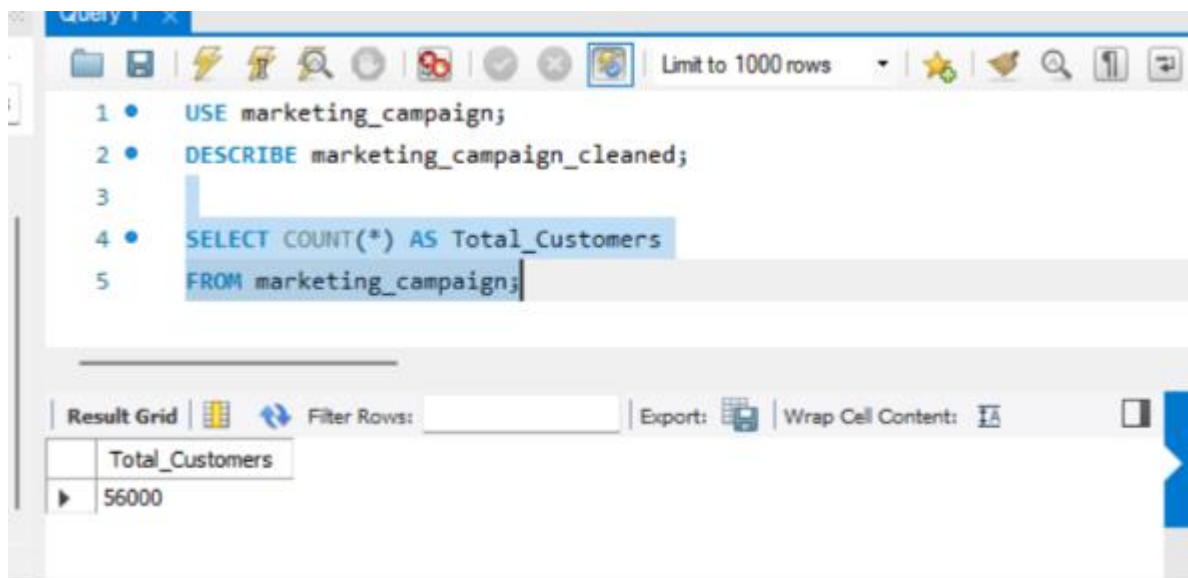
Result 14 x

Report Explanation

The cleaned marketing campaign dataset was imported into MySQL. The table contains customer demographic information, purchase history, campaign response, and derived features such as Age, Customer_Tenure, Children, Total_Spend, and Total_Purchases.

KPI Analysis

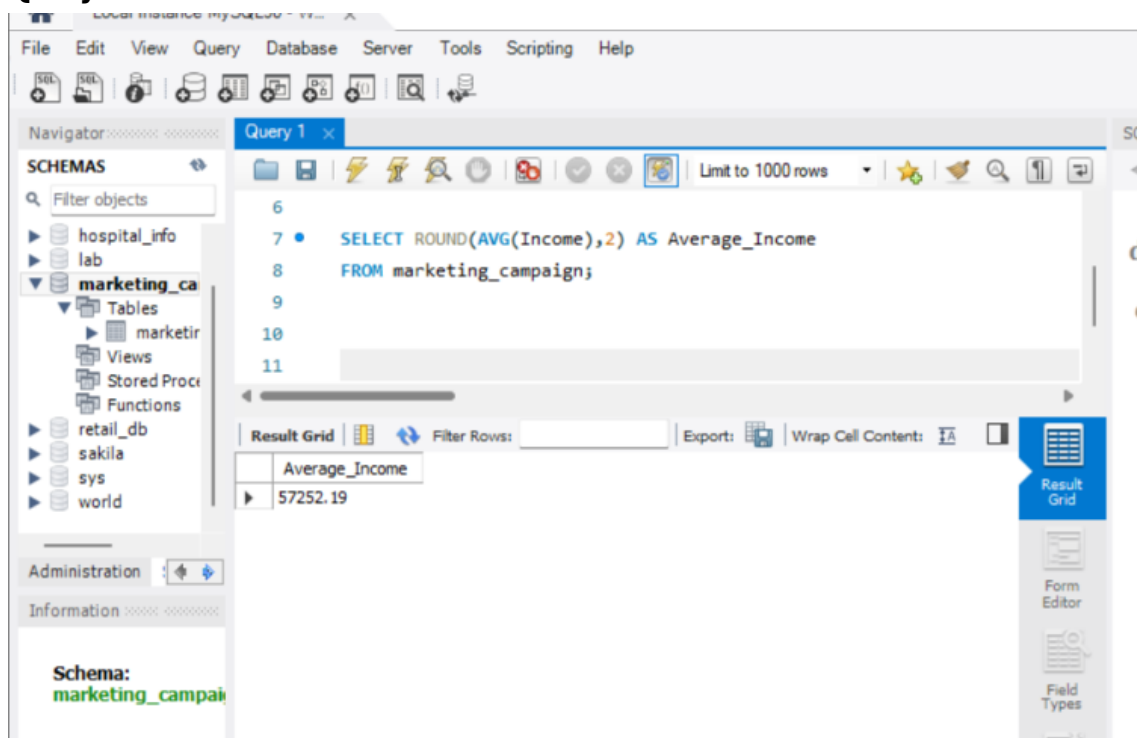
Query 1 –



Explanation:

This query calculates the total number of customer records available in the dataset.

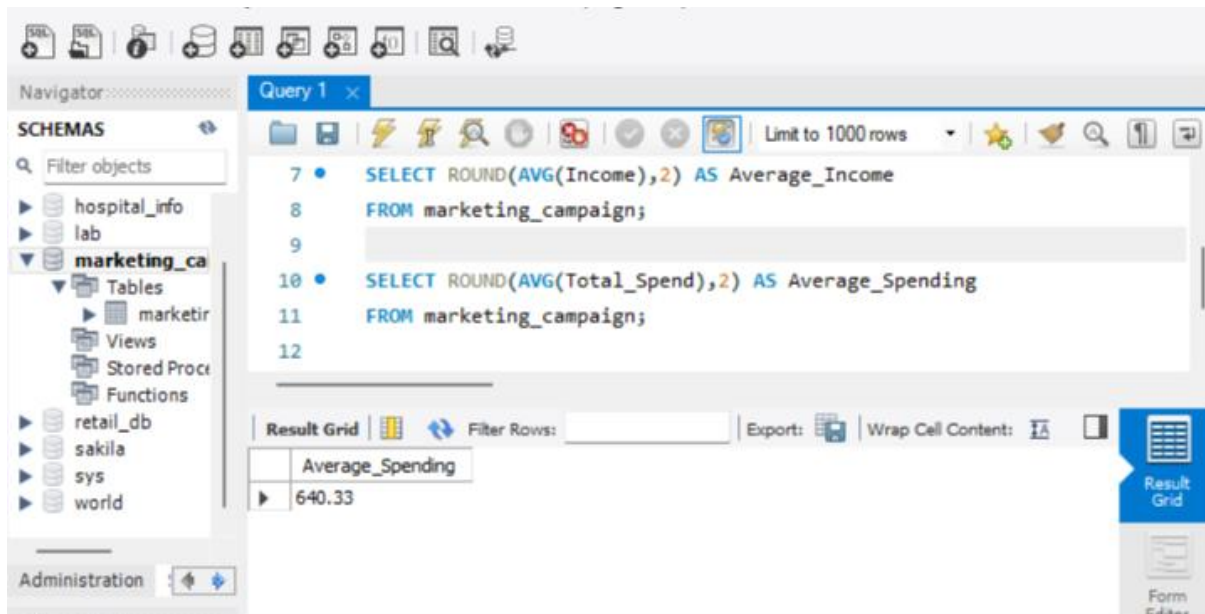
Query 2-



Explanation:

This query calculates the average annual income of customers.

Query 3-



The screenshot shows the MySQL Workbench interface. The 'Query 1' tab is active, displaying two SQL queries. The first query calculates the average income, and the second calculates the average spending. The result grid shows the average spending as 640.33.

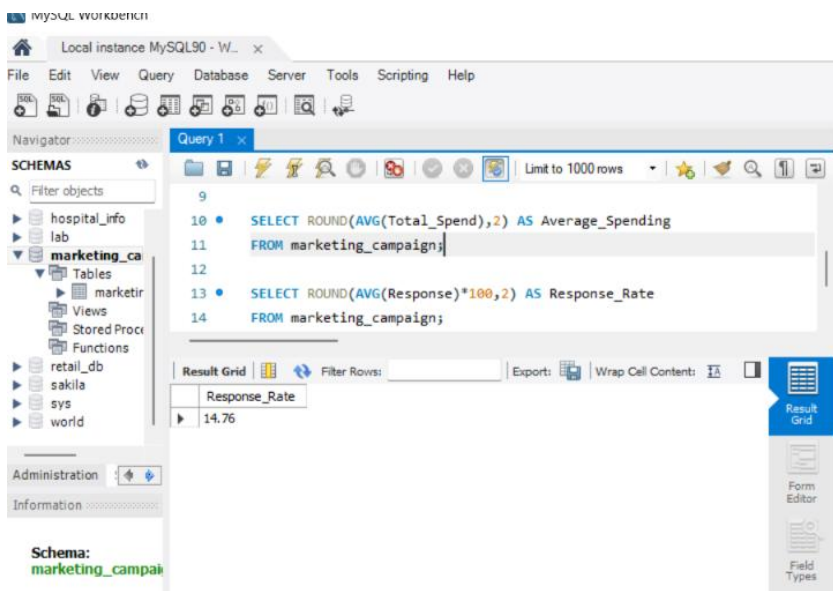
```
7 • SELECT ROUND(AVG(Income),2) AS Average_Income
8   FROM marketing_campaign;
9
10 • SELECT ROUND(AVG(Total_Spend),2) AS Average_Spending
11   FROM marketing_campaign;
12
```

Average_Spending
640.33

Explanation:

This query calculates the average spending of customers across all product categories.

Query 4-



The screenshot shows the MySQL Workbench interface. The 'Query 1' tab is active, displaying two SQL queries. The first query calculates the average spending, and the second calculates the response rate. The result grid shows the response rate as 14.76.

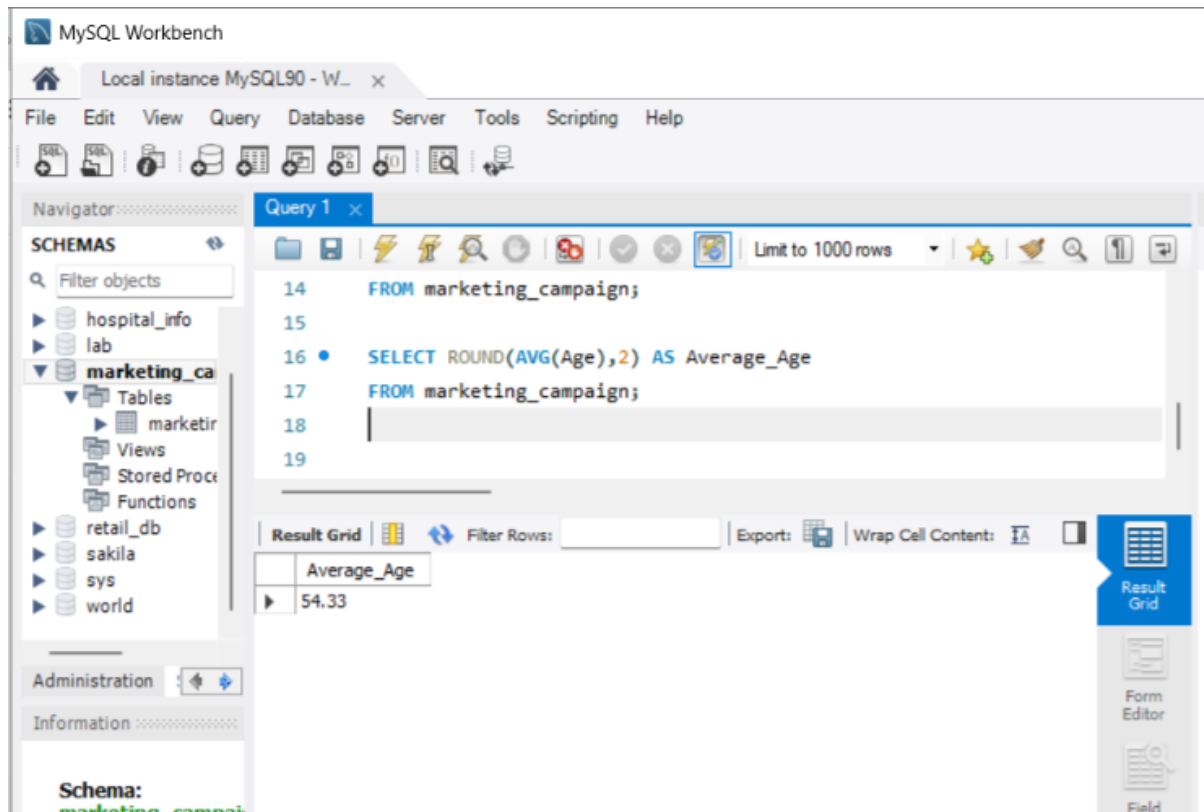
```
9
10 • SELECT ROUND(AVG(Total_Spend),2) AS Average_Spending
11   FROM marketing_campaign;
12
13 • SELECT ROUND(AVG(Response)*100,2) AS Response_Rate
14   FROM marketing_campaign;
```

Response_Rate
14.76

Explanation:

This query calculates the percentage of customers who responded positively to the marketing campaign.

Query 5 –

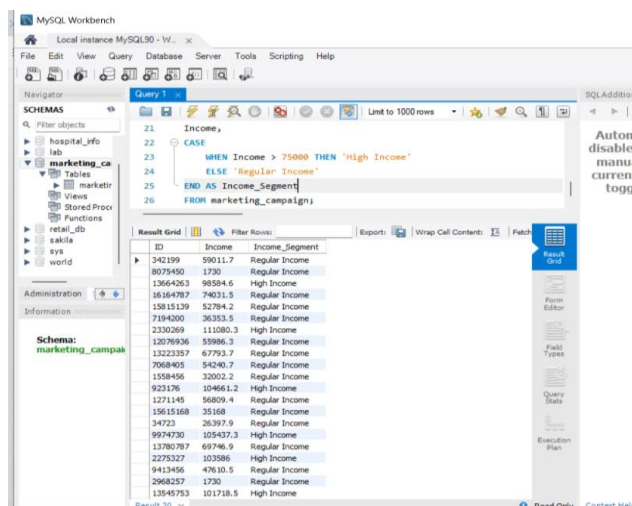


Explanation:

This query calculates the average age of customers.

Rule-Based Segmentation

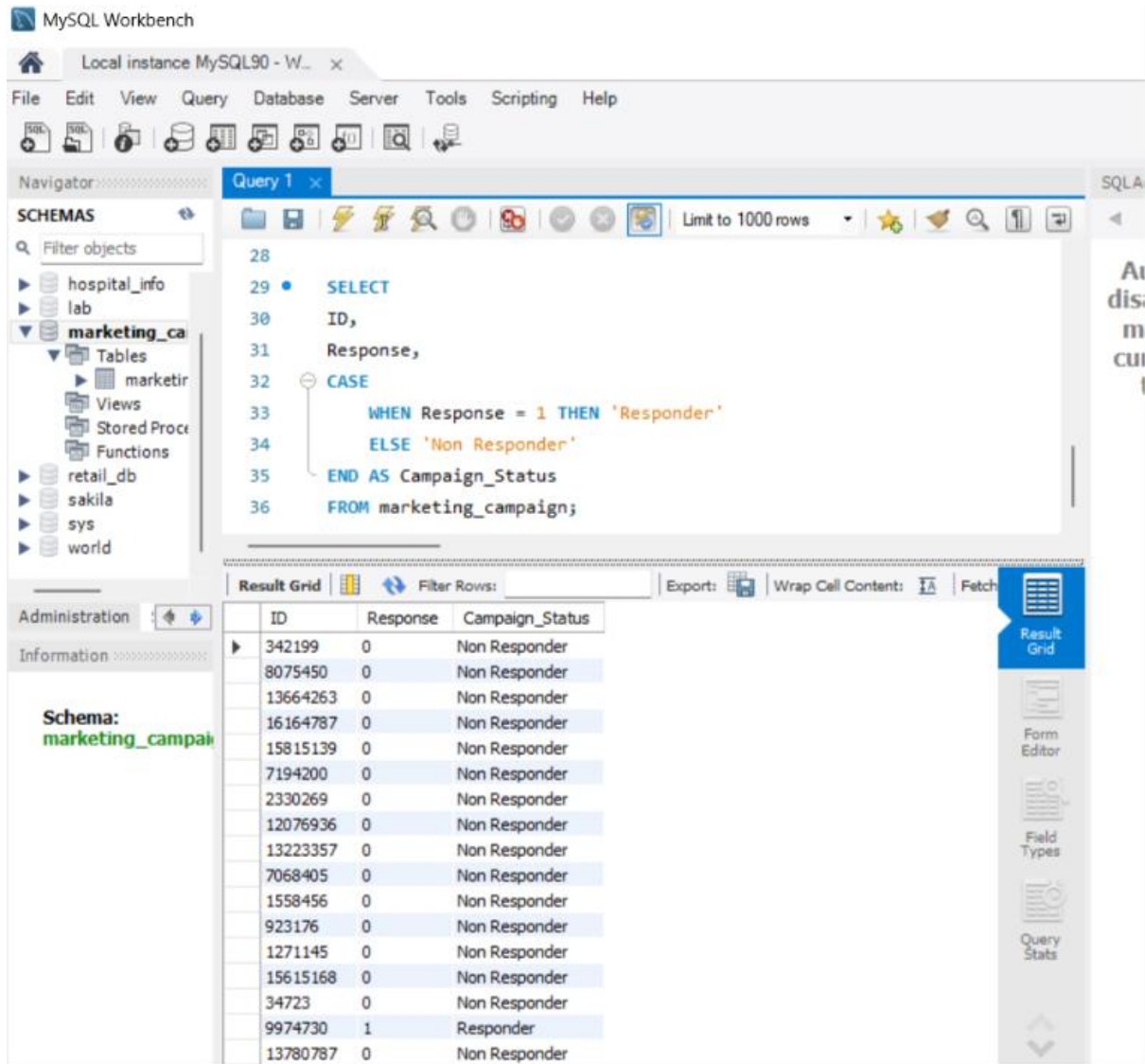
Query 1-High Income Customers



Explanation

This query classifies customers into **High Income** and **Regular Income** segments based on their annual income.

Query 3: Campaign Responders



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'marketing_ca' selected. The main editor shows a SQL query (Query 1) that selects 'ID' and 'Response' from 'marketing_campaign', and uses a CASE statement to assign 'Campaign_Status' as 'Responder' for 'Response' = 1 and 'Non Responder' for 'Response' = 0. The 'Result Grid' at the bottom displays the query results.

```
28
29 • SELECT
30     ID,
31     Response,
32     CASE
33         WHEN Response = 1 THEN 'Responder'
34         ELSE 'Non Responder'
35     END AS Campaign_Status
36 FROM marketing_campaign;
```

ID	Response	Campaign_Status
342199	0	Non Responder
8075450	0	Non Responder
13664263	0	Non Responder
16164787	0	Non Responder
15815139	0	Non Responder
7194200	0	Non Responder
2330269	0	Non Responder
12076936	0	Non Responder
13223357	0	Non Responder
7068405	0	Non Responder
1558456	0	Non Responder
923176	0	Non Responder
1271145	0	Non Responder
15615168	0	Non Responder
34723	0	Non Responder
9974730	1	Responder
13780787	0	Non Responder

Explanation

This query identifies customers who responded to the marketing campaign.

Query 4: High Web Engagement

The screenshot shows a database management tool interface. On the left, a 'SCHEMAS' pane lists various databases, with 'marketing_ca' selected. The main area displays a SQL query in 'Query 1'.

```
37
38 • SELECT
39   ID,
40   NumWebVisitsMonth,
41   CASE
42     WHEN NumWebVisitsMonth > 5 THEN 'High Web Engagement'
43     ELSE 'Low Web Engagement'
44   END AS Web_Engagement
45 FROM marketing_campaign;
```

Below the query, the 'Result Grid' shows the output. The columns are 'ID', 'NumWebVisitsMonth', and 'Web_Engagement'. The results are as follows:

ID	NumWebVisitsMonth	Web_Engagement
342199	4	Low Web Engagement
8075450	3	Low Web Engagement
13664263	3	Low Web Engagement
16164787	4	Low Web Engagement
15815139	6	High Web Engagement
7194200	4	Low Web Engagement
2330269	2	Low Web Engagement
12076936	4	Low Web Engagement
13223357	4	Low Web Engagement
7068405	0	Low Web Engagement
1558456	5	Low Web Engagement
923176	8	High Web Engagement
1271145	4	Low Web Engagement
15615168	0	Low Web Engagement
34723	5	Low Web Engagement
9974730	7	High Web Engagement
13780787	6	High Web Engagement

Explanation

This query classifies customers based on their monthly website visits.

Query 5: Family Customers

The screenshot shows a database management tool interface. On the left, a 'SCHEMAS' pane lists databases: hospital_info, lab, marketing_ca (selected), retail_db, sakila, sys, and world. The 'marketing_ca' database is expanded, showing tables, views, stored procedures, and functions. The main area displays a SQL query in a text editor:

```
46
47 • SELECT
48     ID,
49     Total_Spend,
50 CASE
51     WHEN Total_Spend > 50000 THEN 'High Spender'
52     ELSE 'Regular Spender'
53 END AS Spending_Category
54 FROM marketing_campaign;
```

Below the query editor, the 'Result Grid' shows the output of the query. It has columns for ID, Total_Spend, and Spending_Category. The results show 15 rows, all labeled as 'Regular Spender'.

ID	Total_Spend	Spending_Category
342199	69	Regular Spender
8075450	39	Regular Spender
13664263	1512	Regular Spender
16164787	478	Regular Spender
15815139	330	Regular Spender
7194200	132	Regular Spender
2330269	843	Regular Spender
12076936	98	Regular Spender
13223357	91	Regular Spender
7068405	251	Regular Spender
1558456	128	Regular Spender
923176	2109	Regular Spender
1271145	180	Regular Spender
15615168	301	Regular Spender
34723	78	Regular Spender
9974730	1415	Regular Spender
13780787	108	Regular Spender

Explanation

This query identifies customers with high spending behavior based on a predefined spending threshold.

Aggregation Queries

Query 1: Education-wise Customer count

The screenshot shows a SQL query editor with a query named 'Query 1'. The query is as follows:

```

54 FROM marketing_campaign;
55
56
57 • SELECT
58 Education,
59 COUNT(*) AS Total_Customers
60 FROM marketing_campaign
61 GROUP BY Education
62 ORDER BY Total_Customers DESC;

```

The result grid below the query shows the following data:

Education	Total_Customers
Graduation	22741
PhD	12581
Master	10530
2n Cycle	5966
Basic	4182

Explanation

This query counts the number of customers in each education category.

Query 2: Average Income by Education

The screenshot shows a SQL query editor with a query named 'Query 1'. The query is as follows:

```

61 GROUP BY Education
62 ORDER BY Total_Customers DESC;
63
64 • SELECT
65 Education,
66 ROUND(AVG(Income),2) AS Average_Income
67 FROM marketing_campaign
68 GROUP BY Education
69 ORDER BY Average_Income DESC;

```

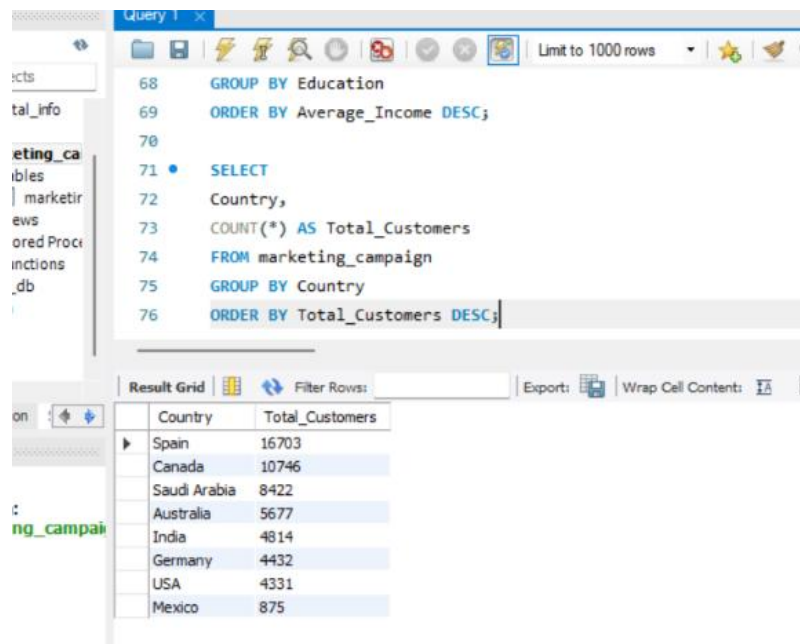
The result grid below the query shows the following data:

Education	Average_Income
Graduation	64675.94
Master	57014.33
PhD	53762.13
2n Cycle	49096.46
Basic	39616.26

Explanation

This query calculates the average income for each education level.

Query 3: Country-wise Customer Count



The screenshot shows a SQL query editor with the following code:

```
68 GROUP BY Education
69 ORDER BY Average_Income DESC;
70
71 • SELECT
72 Country,
73 COUNT(*) AS Total_Customers
74 FROM marketing_campaign
75 GROUP BY Country
76 ORDER BY Total_Customers DESC;
```

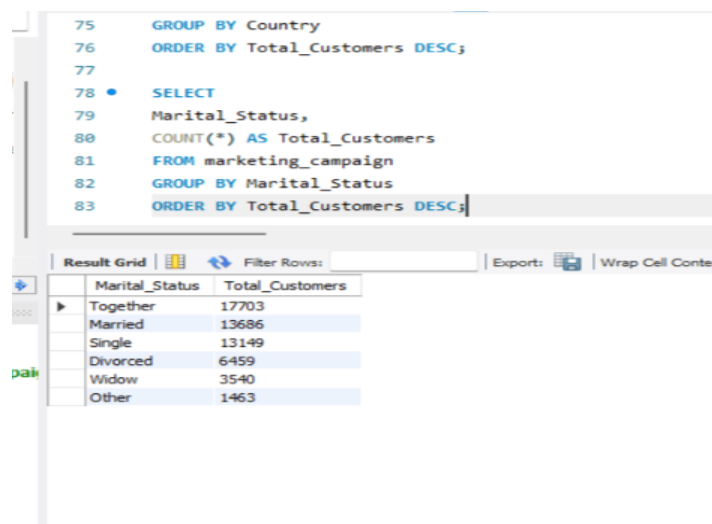
The results are displayed in a grid with the following data:

Country	Total_Customers
Spain	16703
Canada	10746
Saudi Arabia	8422
Australia	5677
India	4814
Germany	4432
USA	4331
Mexico	875

Explanation

This query shows the number of customers from each country.

Query 4: Marital Status Analysis



The screenshot shows a SQL query editor with the following code:

```
75 GROUP BY Country
76 ORDER BY Total_Customers DESC;
77
78 • SELECT
79 Marital_Status,
80 COUNT(*) AS Total_Customers
81 FROM marketing_campaign
82 GROUP BY Marital_Status
83 ORDER BY Total_Customers DESC;
```

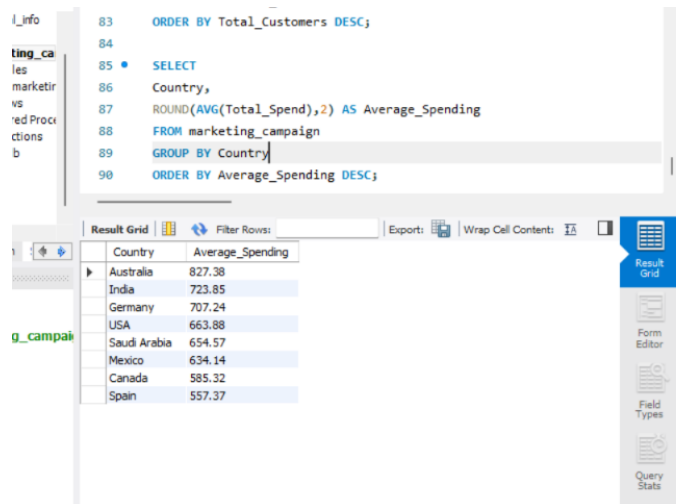
The results are displayed in a grid with the following data:

Marital_Status	Total_Customers
Together	17703
Married	13686
Single	13149
Divorced	6459
Widow	3540
Other	1463

Explanation

This query shows the distribution of customers based on marital status.

Query 5: Average Spending by Country



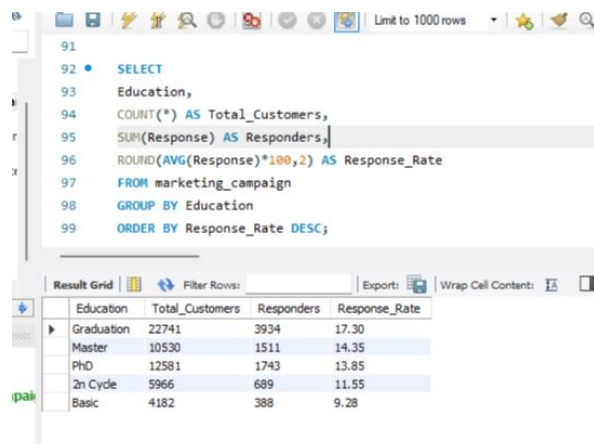
```
83 ORDER BY Total_Customers DESC;
84
85 • SELECT
86 Country,
87 ROUND(AVG(Total_Spend),2) AS Average_Spending
88 FROM marketing_campaign
89 GROUP BY Country
90 ORDER BY Average_Spending DESC;
```

Country	Average_Spending
Australia	827.38
India	723.85
Germany	707.24
USA	663.88
Saudi Arabia	654.57
Mexico	634.14
Canada	585.32
Spain	557.37

Explanation

This query calculates the average customer spending for each country.

Query 6: Campaign Response by Education



```
91
92 • SELECT
93 Education,
94 COUNT(*) AS Total_Customers,
95 SUM(Response) AS Responders,
96 ROUND(AVG(Response)*100,2) AS Response_Rate
97 FROM marketing_campaign
98 GROUP BY Education
99 ORDER BY Response_Rate DESC;
```

Education	Total_Customers	Responders	Response_Rate
Graduation	22741	3934	17.30
Master	10530	1511	14.35
PhD	12581	1743	13.85
2n Cycle	5966	689	11.55
Basic	4182	388	9.28

Explanation

This query calculates the campaign response rate across different education levels.

Window Functions

Query 1: Rank Customers by Total Spending

Query 1

```

101 SELECT
102 ID,
103 Total_Spend,
104 RANK() OVER(ORDER BY Total_Spend DESC) AS Spending_Rank
105 FROM marketing_campaign;

```

Result Grid

ID	Total_Spend	Spending_Rank
5753845	3431	1
6943845	3355	2
13759135	3338	3
16522661	3338	3
13943936	3314	5
10694541	3270	6
14059885	3234	7
2837885	3203	8
2295475	3201	9
16471314	3198	10
4854305	3192	11
16350586	3186	12
12277380	3185	13
4581160	3180	14
10571735	3179	15
9692517	3170	16
10082330	3166	17
11115153	3160	18
11622965	3155	19
16380658	3148	20
2189757	3146	21
10896224	3142	22

Result 30 x Read Only Conte

Explanation

This query ranks customers based on their total spending. Customers with the highest spending receive the highest rank. If two customers have the same spending, they receive the same rank.

Query 2: Dense Rank by Income

Query 2

```

107 SELECT
108 ID,
109 Income,
110 DENSE_RANK() OVER(ORDER BY Income DESC) AS Income_Rank
111 FROM marketing_campaign;

```

Result Grid

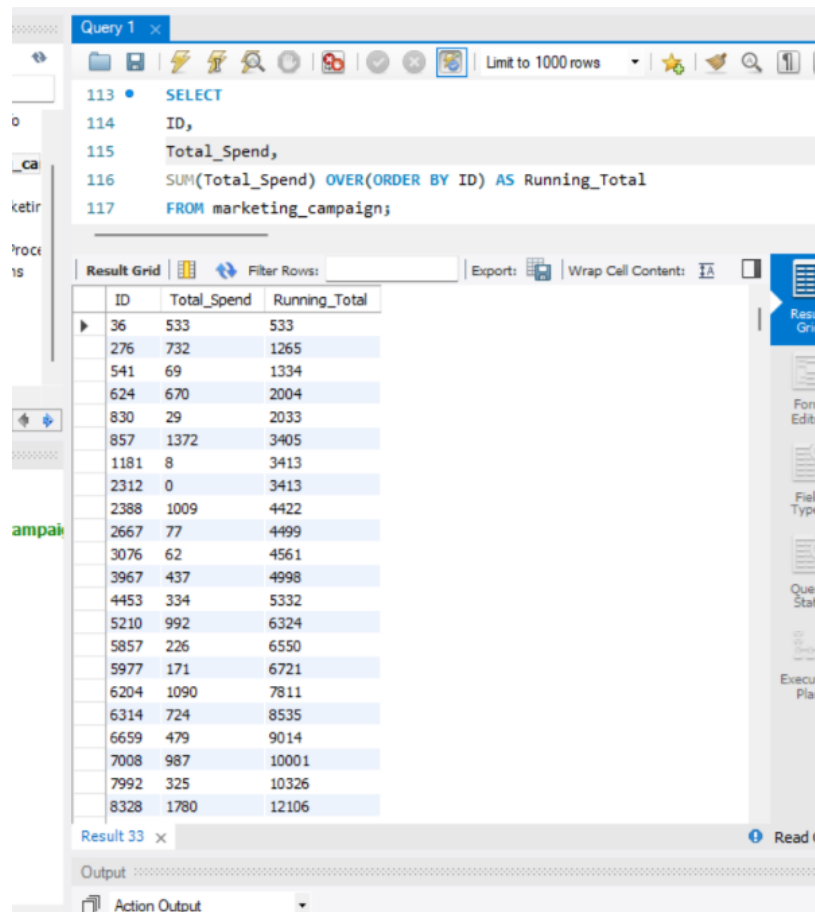
ID	Income	Income_Rank
489682	258027.5	1
11781883	257317.4	2
15094458	235075	3
6633517	231774.1	4
4447366	213924.7	5
10382905	203999.1	6
6595146	197257.7	7
6993170	195998	8
9169447	193497.9	9
15118576	193007.6	10
16489272	189695.9	11
14596919	186727.6	12
10447992	183504.5	13
4585031	179550.9	14
13378458	159700	15
15532379	143793.1	16
5158280	136377.3	17
65938	122953.9	18
5313373	118144.3	19
323473	118107.8	20
6372360	117964	21
110534	117720.8	22

Result 32 x Read Only Conte

Explanation

This query assigns a dense rank to customers based on their income. Unlike RANK(), DENSE_RANK() does not skip rank numbers when there are ties.

Query 3: Running Total of Spending



Query 1 x

```
113 • SELECT
114 ID,
115 Total_Spend,
116 SUM(Total_Spend) OVER(ORDER BY ID) AS Running_Total
117 FROM marketing_campaign;
```

ID	Total_Spend	Running_Total
36	533	533
276	732	1265
541	69	1334
624	670	2004
830	29	2033
857	1372	3405
1181	8	3413
2312	0	3413
2388	1009	4422
2667	77	4499
3076	62	4561
3967	437	4998
4453	334	5332
5210	992	6324
5857	226	6550
5977	171	6721
6204	1090	7811
6314	724	8535
6659	479	9014
7008	987	10001
7992	325	10326
8328	1780	12106

Result 33 x

Output

Action Output

Explanation

This query calculates the cumulative spending of customers in ID order using a running total.

Business Insights

- Customers with higher income generally show higher purchasing power.
- High spenders contribute significantly to total revenue.
- Campaign responders represent potential loyal customers for future marketing campaigns.

- Customers with high website engagement are more likely to interact with digital campaigns.
- Family customers can be targeted with bundled or family-oriented offers.
- Education and country-wise analysis helps identify customer segments with better campaign performance.

This project analyzed customer demographics, purchasing behavior, and campaign responses using Python, SQL, and Streamlit. Data was cleaned and transformed using Pandas, imported into MySQL for SQL-based analysis, and visualized through an interactive dashboard. Business insights generated from KPI analysis, rule-based segmentation, aggregation queries, and window functions can help organizations improve customer targeting and marketing strategies.